

NexysA7 UART

By: Ben Robles

Cal Poly Pomona
Summer 2026

Design:

Idle	Start	D0	D1	D2	D3	D4	D5	D6	D7	Stop	Idle
1	0	1	0	0	0	0	0	1	0	1	1

Variable baud rate 8-N-1 UART protocol at 100 MHz system clock

Modules:

- baud_tick.v
- tx.v
- rx_sync.v
- rx.v
- debounce.v
- top_uart.v

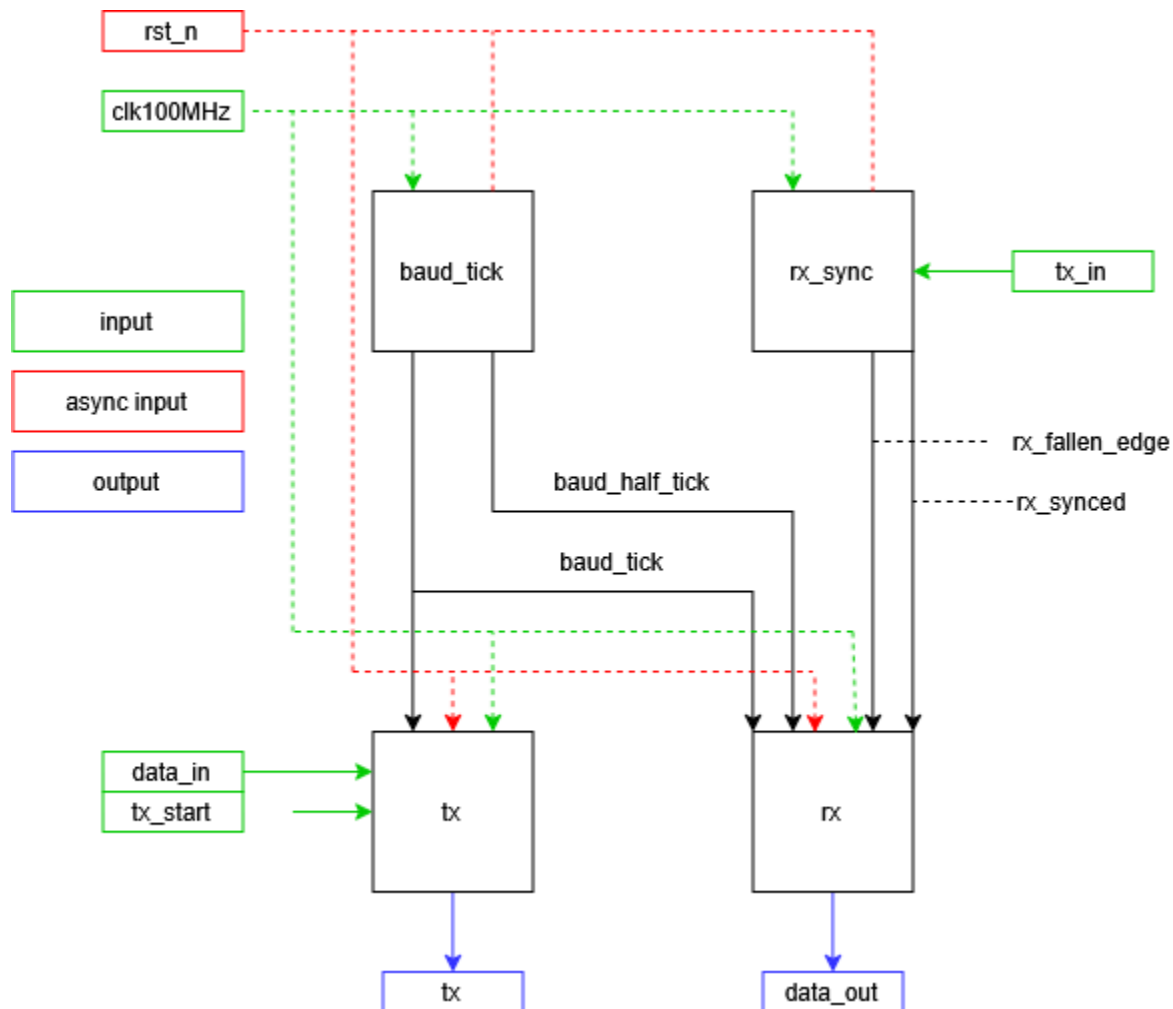
Explanation:

The design is implemented with an asynchronous reset

- Primary inputs: 100 MHz clock, rst_n, data_in, tx_start, tx_in
- Primary outputs: tx_out, data_out
- Primary Modules:
 1. baud_tick.v generates a variable baud tick for every full and a half bit period

2. tx.v uses a Moore FSM to drive tx each frame (10 bit periods) through Idle → Start → Data[0] → ... → Data[7] → Stop → Idle
 3. rx.sync.v syncs the asynchronous tx_input to the system clock and detects falling edges (start bit)
 4. rx.v uses a Moore FSM to read the tx_input frame in a similar way to tx.v. Each bit is sampled at the halfway tick of its bit period to compensate for clock mismatch
- Implementation Modules:
 1. debounce.v debounces the input button and generates a pulse for tx_start
 2. top_uart.v instantiates each module and wires them together. tx and rx go to the PMOD I/O JA1 and JA2 respectively. Tx_start and rst_n are controlled by btnc and ~btnd. All remaining outputs are displayed on the LEDs

Block Diagram:



*excludes debounce & top level

Resources:

To build off of my digital circuit design verilog lab, I asked ChatGPT to generate a lab style write-up based on my professors previous labs. These were not followed strictly and mainly used as a starting point. AI was used for troubleshooting and learning but all code was written and verified by hand.

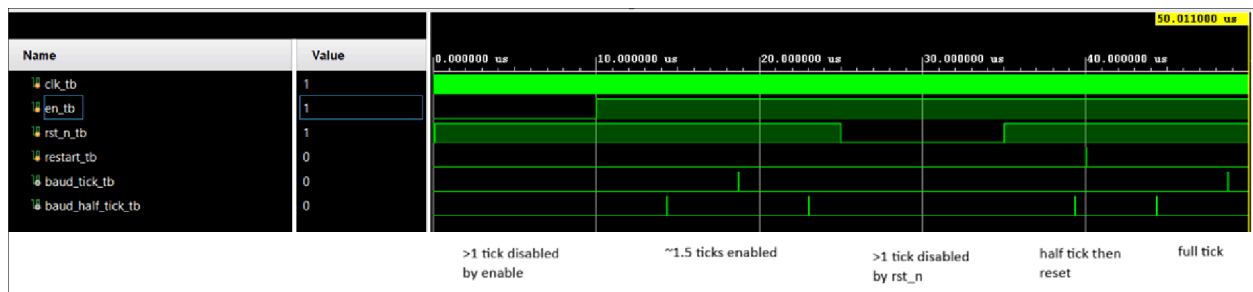
Simulation:

Testbenches for each module (rx_tb uses a loopback simulation)

See waveforms folder for higher resolution annotated images

baud_tick.v:

full tick = system clock / baud rate = 100 MHz / 115200 = 8.68 μ s



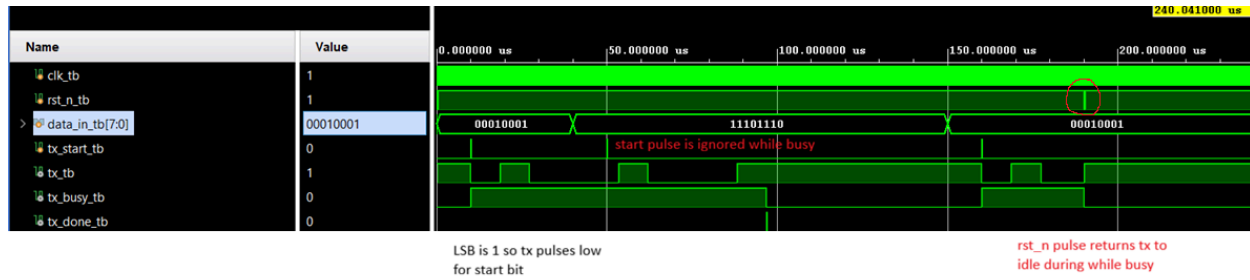
First full tick:



Full tick after final reset:



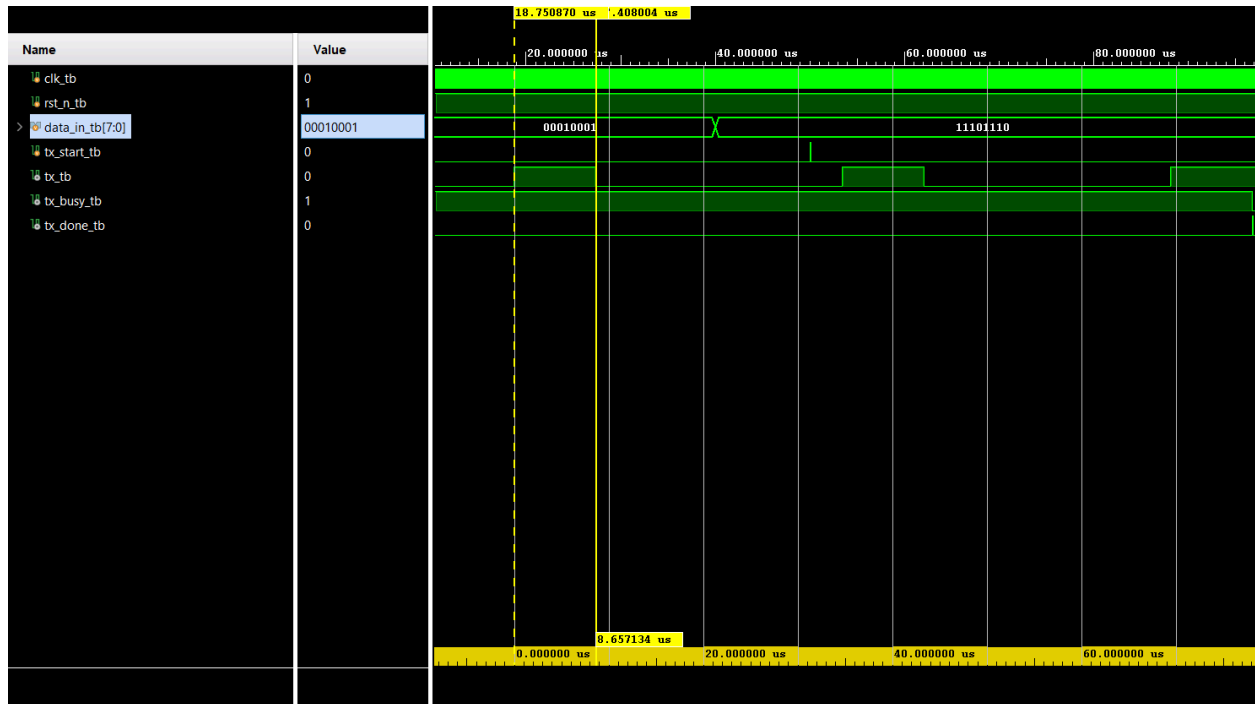
tx_tb.v:



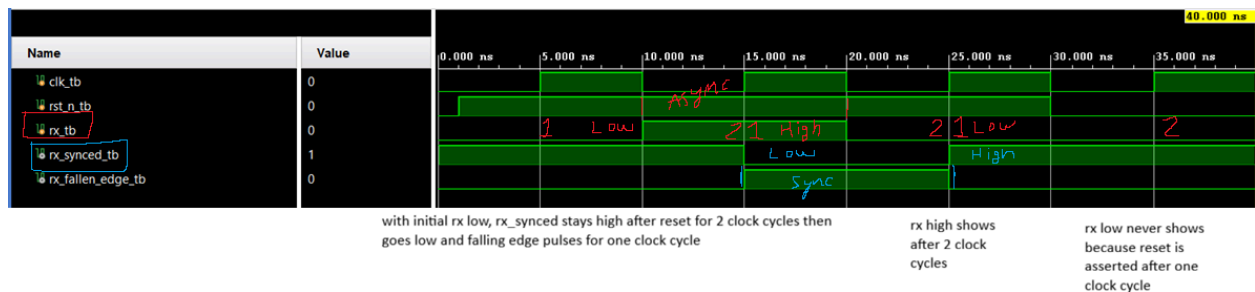
A full frame is 10-bits or 86.8 μ s:



1 bit period:

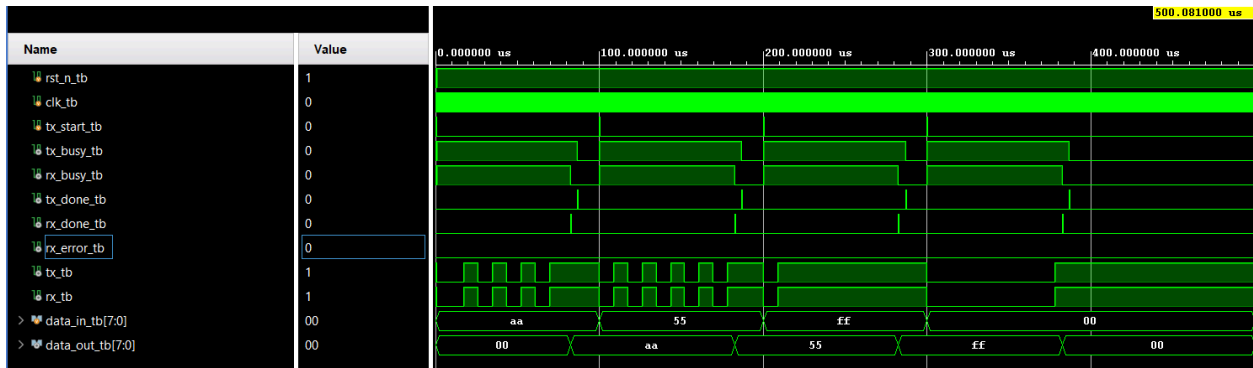


rx_sync_tb.v:



uart_tb.v:

Loopback test: data_in = data_out



One frame:



Implementation:

Resource Utilization Report:

Name	Slice LUTs (63400)	Slice Registers (126800)	F7 Muxes (31700)	Slice (15850)	LUT as Logic (63400)	Bonded IOB (210)	BUFGCTRL (32)
> N top_uart	78	68	1	30	78	27	1

Timing Report:

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 5.851 ns	Worst Hold Slack (WHS): 0.166 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 78	Total Number of Endpoints: 78	Total Number of Endpoints: 69
All user specified timing constraints are met.		

Demo:

<https://youtu.be/FzElmZiW2JE>

To demo the top level implementation I used an ESP32 with the Arduino IDE to create an extremely basic UART script

Conclusion:

I've used UART for years but never knew exactly how it worked. This was my first independent project in my pursuit of Verilog and I intend to use it as a framework for future projects. The project took awhile but I really enjoyed it, especially the rtl design.

Current Revision: 2

(Revision 1: Ben Robles 8/12)

(Revision 2: Ben Robles 8/20)